WebScanPro

Web Application Security Testing Tool

Project Overview

WebScanPro is a custom-built web application security scanner developed for educational purposes. The tool progressively builds scanning modules over four weeks, targeting DVWA (Damn Vulnerable Web Application) hosted locally via XAMPP.

Project Focus Areas

WebScanPro covers the following areas:

- Setting up vulnerable web applications locally (DVWA + XAMPP)
- Building an authenticated web crawler to discover pages and forms
- Extracting forms, input fields, and CSRF tokens automatically
- Implementing automated SQL Injection detection
- Implementing automated Cross-Site Scripting (XSS) detection
- Generating structured vulnerability output reports

Technology Stack

Language	Python 3
Libraries	Requests, BeautifulSoup, urllib.parse, JSON
Target App	DVWA (Damn Vulnerable Web Application)
Server	XAMPP (Apache + MySQL)
IDE	VS Code

Week 1 – Project Initialization & Setup

Objectives

- Define project goals and expected outcomes
- Set up DVWA locally using XAMPP
- Explore application structure and vulnerabilities
- Prepare environment for automated scanning

Environment Setup

1. Install XAMPP

- Download and install XAMPP on Windows
- Start Apache (Port 443 / 8080) and MySQL (Port 3306)
- Verified services running in XAMPP Control Panel v3.3.0

Screenshot: XAMPP Control Panel v3.3.0 – Apache and MySQL running (green)

2. Install & Configure DVWA

- Downloaded DVWA and placed inside C:\xampp\htdocs\
- Accessed via http://localhost/dvwa – clicked Create / Reset Database
- Login: admin / password – Security Level set to LOW

Screenshot: DVWA installation and database reset

- DVWA Security Level set to LOW

Screenshot: DVWA Security Level set to LOW

3. PHP Configuration

PHP parameters enabled to meet DVWA requirements: allow_url_include, allow_url_fopen, GD library. Apache restarted and verified via phpinfo().

Application Exploration

The following DVWA elements were analysed: login functionality, form-based modules, GET/POST requests, input field behaviour, and hidden CSRF tokens.

Initial Security Observations

Vulnerabilities identified during exploration (no exploitation at this stage):

- SQL Injection – unsanitised input fields
- Cross-Site Scripting (XSS) – reflected and stored
- Weak Authentication – default credentials (admin/password)
- Improper Input Validation – across multiple modules

Week 1 – Outcome

- ✓ Local testing environment successfully configured
- ✓ DVWA deployed, initialised, and accessible
- ✓ Required PHP dependencies resolved
- ✓ Application structure and potential vulnerabilities understood
- ✓ Environment fully prepared for crawler development in Week 2

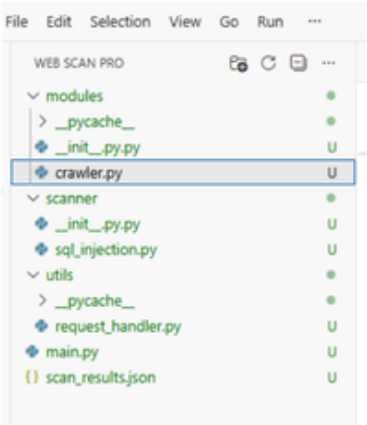
Week 2 – Target Scanning Module (Web Crawler)

Objectives

- Develop an authenticated web crawler
- Discover internal pages and navigation links
- Extract forms and input fields automatically
- Structure and save metadata to JSON

Project Structure

```
Web_scan_pro/

├── main.py

├── modules/

│   ├── __init__.py

│   └── crawler.py

├── utils/

│   ├── __init__.py

│   └── request_handler.py

└── scan_results.json
```



howing project folder structure

Handler Module

Screenshot: VS Code showing WebScanPro project folder structure

Request Handler Module (request_handler.py)

Responsibilities:

- Managing persistent HTTP session using requests.Session()
- Extracting hidden CSRF token (user_token) from the login page HTML
- Submitting login credentials programmatically via POST
- Maintaining authenticated session across all crawler requests

Web Crawler Module (crawler.py)

Responsibilities:

- Crawling DVWA pages recursively from the base URL
- Discovering and following all internal href links
- Extracting all <form> elements – action URL and HTTP method
- Extracting all input fields per form – name and type attributes
- Storing structured metadata in a results list
- Saving final results to scan_results.json

Technologies Used

- BeautifulSoup – HTML parsing and element extraction
- urllib.parse – URL resolution and link joining
- JSON module – structured output serialisation

Crawler Workflow

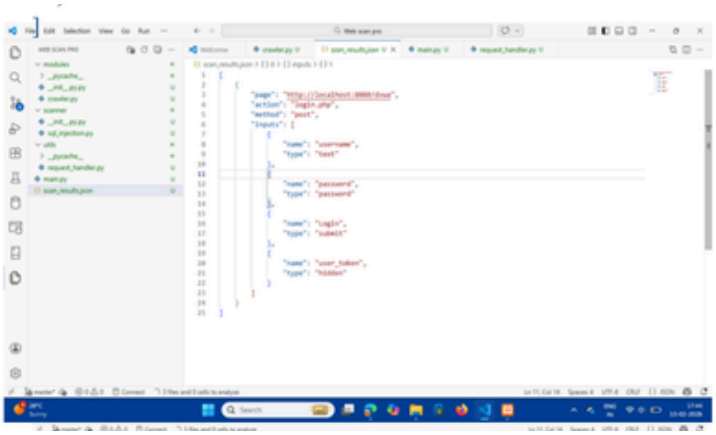
Step 1	Start at base DVWA URL
Step 2	Send authenticated GET request with session cookie
Step 3	Parse HTML response using BeautifulSoup
Step 4	Extract all <form> elements
Step 5	Capture form action, method, and all input fields
Step 6	Store metadata in results list
Step 7	Recursively follow all internal href links
Step 8	Save complete results to scan_results.json

Console Output & JSON Results

Console Output:

```
[*] Web Scan Pro Starting...
[+] Login successful!
[*] Starting crawl at http://localhost:8080/dvwa
[+] Found page: ...
[+] Scan results saved to scan_results.json
```

Screenshot: Crawler running – console output showing login and crawl progress



Screenshot: scan_results.json showing extracted form metadata

JSON Output Structure

```
[
  {
    "page":   "http://localhost:8080/dvwa",
    "action": "login.php",
    "method": "post",
    "inputs": [
      { "name": "username",   "type": "text"     },
      { "name": "password",   "type": "password" },
      { "name": "Login",      "type": "submit"   },
      { "name": "user_token", "type": "hidden"   }
    ]
  }
]
```


Week 2 – Achievements

- ✓ Authenticated crawler built with full CSRF token support
- ✓ All interactive form elements and input fields extracted
- ✓ Structured metadata saved to scan_results.json
- ✓ Foundation ready for vulnerability injection modules in Week 3

Week 3 – SQL Injection (SQLi) Scanner

Objective

Design and implement an automated SQL Injection scanner to detect SQL vulnerabilities in DVWA by injecting known payloads and analysing server responses for database output or error messages.

Tools Used

Language	Python 3
Libraries	Requests, BeautifulSoup
Target	DVWA – SQL Injection module
Server	XAMPP (Apache + MySQL)
IDE	VS Code
Output	results.txt

Working Mechanism

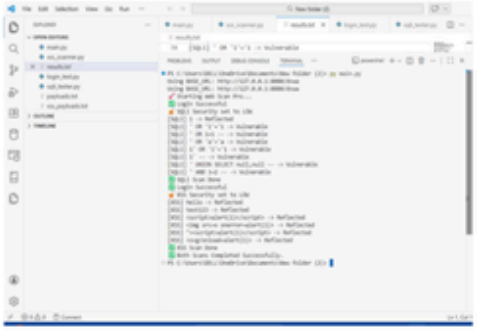
Step 1	Establish session using requests.Session()
Step 2	Extract CSRF token from DVWA login page
Step 3	Login with valid credentials (admin / password)
Step 4	Set DVWA security level to LOW
Step 5	Inject SQL payloads into the ID input field via GET
Step 6	Analyse server response for DB output or error strings
Step 7	Mark endpoint as Vulnerable if abnormal output detected
Step 8	Store all results in results.txt

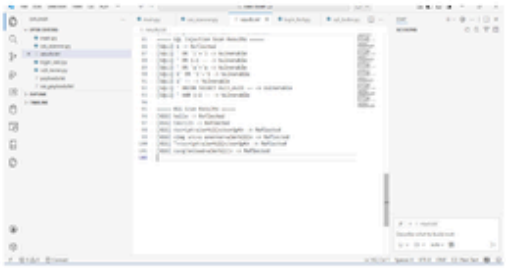
Detection Strategy

Two complementary detection techniques were applied:

- Content-based detection: checks response for 'First name:' — confirms DB data was returned
- Error-based detection: scans response for known MySQL error strings

Terminal Output & Scan Results





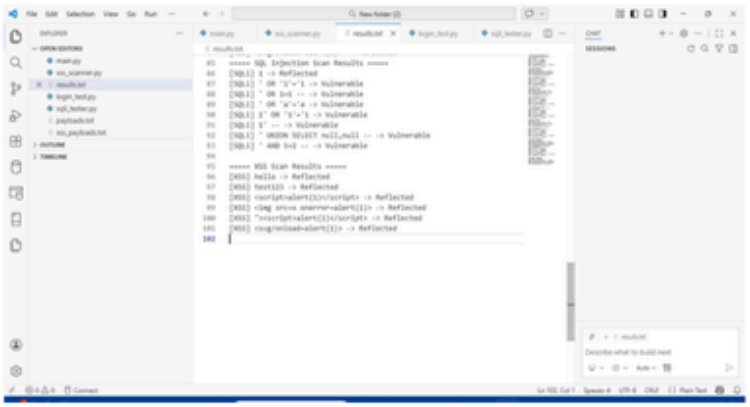
DVWA Login Page - Security Level set to LOW - Terminal showing SQLi Scan Output - results.txt showing SQLi results

Screenshot: Terminal showing SQLi scanner running – scan output and login confirmation

Output Sample:-

Example output:

- [SQLi] 1 -> Safe
- [SQLi] ' OR '1'='1 -> Vulnerable
- [SQLi] ' UNION SELECT null,null -- -> Vulnerable



Screenshot: results.txt showing full SQLi and XSS scan results

Payload Results Table

Payload	Result
`1`	**Safe**
`' OR '1'='1`	**Vulnerable**
`' OR 'a'='a`	**Vulnerable**
`' OR 1=1 --`	**Vulnerable**
`' UNION SELECT null,null --`	**Vulnerable**
`' AND 1=2 --`	**Vulnerable**

Week 3 – Achievements

- ✓ SQL Injection scanner successfully implemented and tested on DVWA
- ✓ Dual detection strategy ensures reliable and accurate results
- ✓ All vulnerable payloads correctly identified and flagged
- ✓ Scan output saved automatically to results.txt
- ✓ Module integrated into the WebScanPro project codebase

Week 4 – Cross-Site Scripting (XSS) Scanner

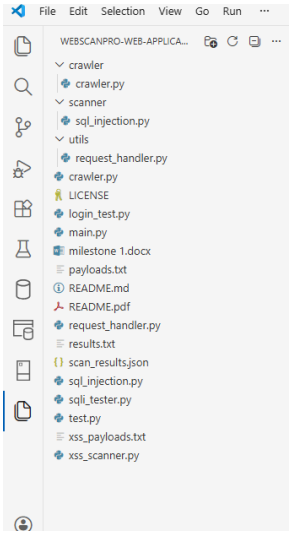
Objective

Implement an automated XSS scanner to detect reflected Cross-Site Scripting vulnerabilities in DVWA. The scanner injects JavaScript payloads into input fields and checks whether the payload is reflected verbatim in the server response.

Tools Used

Language	Python 3
Libraries	Requests, BeautifulSoup
Target	DVWA – XSS (Reflected) module
Server	XAMPP (Apache + MySQL)
IDE	VS Code
Output	results.txt

File structure:



Working Mechanism

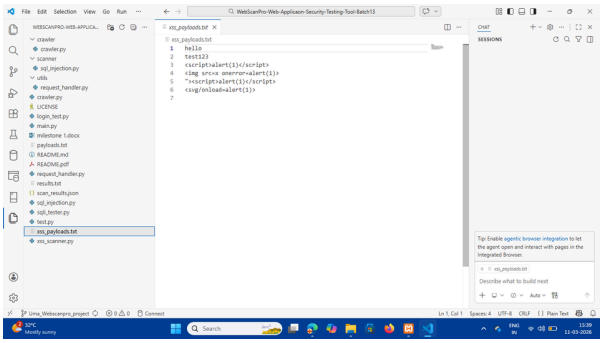
Step 1	Create HTTP session using requests.Session()
Step 2	Log into DVWA with valid credentials
Step 3	Set DVWA security level to LOW
Step 4	Send XSS payloads via the name input field (GET)
Step 5	Check if injected payload appears verbatim in response body
Step 6	Mark as 'Reflected' if payload found in response
Step 7	Save all results to results.txt

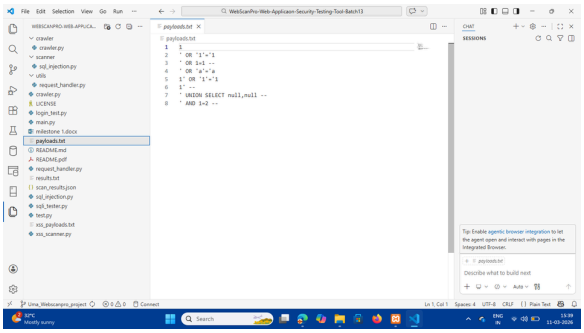
Detection Logic

```
if payload in response.text:

    status = "Reflected"

else:

    status = "Not Reflected"
```

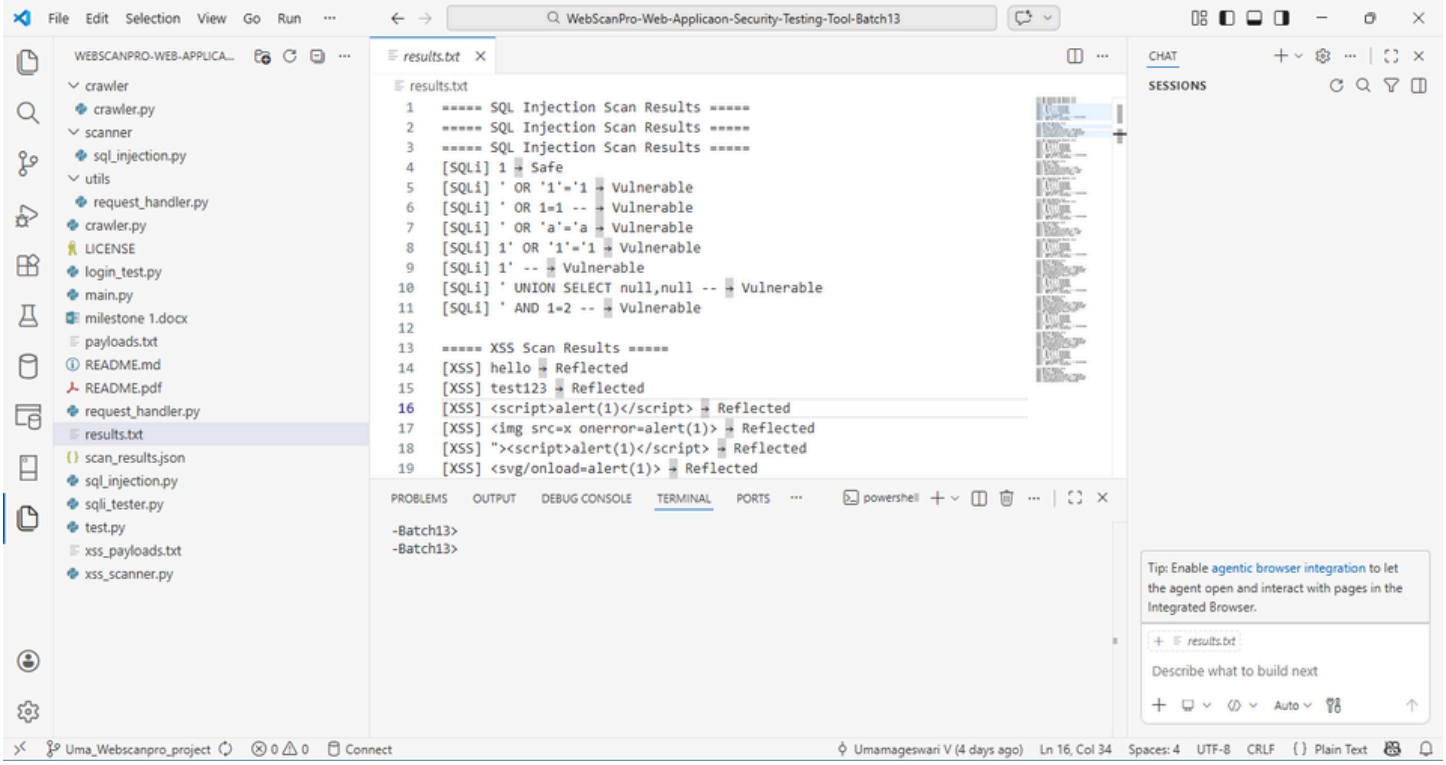
If the injected script appears verbatim in the server response body, the endpoint is confirmed as vulnerable to reflected XSS.

PAYLOADS :





DVWA XSS Module & Scan Output



Screenshot: Terminal and results.txt showing XSS scan output

Payload Results Table

Payload	Result
`hello`	**Not Reflected**
`test123`	**Not Reflected**
`<script>alert(1)</script>`	**Reflected**
`<img src=x onerror=alert(1)>`	**Reflected**
`<svg/onload=alert(1)>`	**Reflected**
`<ScRipT>alert(1)</ScRipT>`	**Reflected**
`<img src=x oNeRrOr=alert(1)>`	**Reflected**

Challenges Faced

Issues encountered and resolved during Week 4:

- CSRF token handling – token must be re-extracted fresh before each request
- Login failure debugging – incorrect headers / stale cookies caused 403 errors
- Unicode encoding errors – non-ASCII payload characters required explicit encoding
- Detection tuning – partial string matches initially caused false positives

Week 4 – Achievements

- ✓ XSS scanner successfully implemented and tested on DVWA
- ✓ Reflected XSS correctly detected for all script-based payloads
- ✓ Safe inputs correctly marked as Not Reflected
- ✓ Scan results saved automatically to results.txt
- ✓ Full SQLi + XSS combined scan pipeline now operational

Learning Outcomes

Skills Acquired Across Weeks 1–4:

- Session handling and cookie management in web applications
- CSRF token extraction and replay in automated HTTP requests
- Recursive web crawling and HTML parsing with BeautifulSoup
- Automated SQL Injection detection using payload + response analysis
- Automated XSS detection via response reflection checking
- Structured JSON and text-based vulnerability report generation
- Modular Python project architecture for security tooling
- Debugging authentication flows, encoding issues, and HTTP error codes

Conclusion

WebScanPro has been successfully built across four progressive weeks — from environment setup to a fully operational multi-module vulnerability scanner. Each week introduced a new layer of capability, resulting in an automated tool that can log in, crawl, and test for both SQLi and XSS vulnerabilities.

Week 1	Environment setup – XAMPP, DVWA installation & PHP configuration
Week 2	Web crawler – authenticated session, form extraction, JSON output
Week 3	SQLi scanner – payload injection, dual detection, results.txt output
Week 4	XSS scanner – reflected injection detection, combined scan pipeline

- ✓ Automated login and session management implemented
- ✓ Web crawler successfully extracts forms and inputs from DVWA
- ✓ SQL Injection vulnerabilities detected and logged
- ✓ Reflected XSS vulnerabilities detected and logged
- ✓ Full vulnerability report file generated automatically
- ✓ All milestones for Weeks 1 through 4 completed successfully